

Towards Utilizing Cached Context for Faster and Smarter Code Agents

Frank Chen

CMU-CS-25-139

August 2025

Computer Science Department
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

Thesis Committee

Prof. Rashmi K. Vinayak, Chair
Prof. Zhihao Jia

*Submitted in partial fulfillment of the requirements
for the degree of Master of Science.*

Keywords: LLM, Agent, Cache, Retrieval Augmented Generation, Software Engineering

Abstract

Recent advances in Large Language Models (LLMs) have enabled the development of coding agents that can autonomously perform tasks such as code generation, debugging, patch validation, etc. Industry-proprietary systems (e.g., Colab AI, Claude Code, Cursor Agent) and open-source frameworks (e.g., Gemini Cli, SWE-Agent, AutoCodeRover, Agentless) have demonstrated both practicality and popularity in real-world software engineering workflows. Despite these successes, existing agentic frameworks face two common challenges: providing accurate and complete context information and ensuring low-latency patch generation under heavy workloads. Although prior work has proposed partial solutions addressing either context retrieval or latency, little attention has been paid to the joint optimization of both aspects. Ideally, joint optimization should enhance both performance and speed without incurring additional cost, which is particularly critical in modern fast-paced, iterative software engineering environments. This thesis investigates integrating existing state-of-the-art approaches to these challenges, specifically RepoGraph for smart context retrieval on the frontend and CacheBlend for fast inference on the backend. To verify feasibility of an integration, we then evaluate performance of the frontend across multiple LLMs under realistic code-agent scenarios, and measure the latency improvement of the backend against systems such as vLLM and SGLang on trace generated by frontend under the same benchmark. The results highlight the trade-offs between cost, efficiency, and performance and argue for the necessity of integrated solutions that achieve balance between the factors mentioned. Finally, we suggest a preliminary design for an end-to-end system that combines the benefits of RepoGraph and CacheBlend via a simple adapter module, along with optimizations in the original algorithms. Overall, the findings suggest promising directions toward building a robust coding agent that is both fast and high-performing.

Acknowledgments

Contents

1	Introduction	1
2	Background	4
2.1	Position-Independent KV Cache Reuse	4
2.2	Repository-Aware Context Retrieval	5
2.3	Motivation for Integration	6
3	System Design	8
3.1	Cached Context	8
3.2	System Architecture	9
3.2.1	Backend: API Server with CacheBlend	9
3.2.2	Adapter Module	10
3.2.3	Frontend: RepoGraph + Agentless	10
3.3	Backend Enhancements	11
3.3.1	Non-Contiguous Cache Hit Support	11
3.3.2	Enhanced Memory Allocation and Locking	11
3.3.3	Per-Request Profiling Infrastructure	12
3.3.4	Request Logging and Replay	13
3.4	Frontend Integration Details	14
3.4.1	Context Truncation	14
3.4.2	Local Store for Precomputed Data	14
4	Evaluations	15
4.1	Latency Evaluation for KV Reuse	15
4.1.1	Setup	15
4.1.2	Results	16
4.1.3	Analysis: Small-Request Speedup Variance	19
4.2	Performance Evaluation for Context Retrieval	24
4.2.1	Setup	24
4.2.2	Results	25
	Bibliography	27

List of Figures

3.1	End-to-end system architecture.	9
3.2	Example profiler output: per-operation latency timeline during a CacheBlend-enabled serving session (chunk lower bound 128 tokens). Each point is a single profiled operation, color-coded by type. The clustering of <code>retrieve_layer</code> (orange) at 25–75 ms reflects the fixed per-step overhead analyzed in Section 4.1.3.	13
4.1	3D surface plot of TTFT speedup vs. cache hit ratio and total prompt tokens (outliers removed). The fitted plane shows that hit ratio is the primary driver of speedup, with a breakeven point at ~39% hit ratio.	17
4.2	TTFT speedup distribution by prompt token-count bucket (outliers removed). Violin plots show the probability density; box plots mark the median and interquartile range.	18
4.3	Distribution of per-request TTFT speedup across 537 requests in the SWE-bench Verified-mini trace.	19
4.4	Chunked-prefill fragmentation rate by prompt token-count bucket. Requests are classified as non-fragmented (1 prefill step, 4 ops) or fragmented (9 prefill steps, 36 ops). Small requests (<2k tokens) exhibit the highest fragmentation rate at 45%.	20
4.5	Per-call <code>retrieve_layer</code> cost broken down by phase group (blend vs. no-blend). GPU Onload dominates at 91% of blend’s per-call time, with a $1.93\times$ ratio over no-blend due to gap-aware KV recomputation.	21
4.6	TTFT speedup vs. total prompt tokens (log scale), colored by fragmentation class. Non-fragmented requests (blue circles) cluster tightly below breakeven for small prompts; fragmented requests (orange diamonds) exhibit the full variance spread. Both classes converge for long prompts.	22
4.7	Resolve rate distributions for SWE-bench Verified-mini evaluation runs across different models.	26
4.8	Resolve rate distributions for SWE-bench Verified-mini evaluation runs across different blending methods, served model is Devstral Small.	26

Introduction

Recent studies on AI coding agents have proven the efficacy of using Large Language Models (LLMs) to handle coding tasks such as code generation and debugging. To apply LLMs in daily software engineering scenarios, both industry and academia have proposed many agentic frameworks. Industry-proprietary examples include Google’s Colab AI [SKS25], Claude Code [Ant25], and Cursor Agent [LIB25]. There is a continuous and rapid growth of user bases for these frameworks, reflecting their perceived utility in real-world software engineering tasks. Besides the proprietary frameworks, there are also many open-source implementations from both academia and industry, including Gemini Cli [MS25], SWE-Agent [Yan+24], AutoCodeRover [Zha+24c], and Agentless [Xia+24]. These frameworks enable existing LLMs to achieve high task success rates when prompted with clear instructions and sufficient code context.

A common characteristic of these agents is that they issue many requests to the LLM during a single task. Each request often carries a long prompt: repository structure information, multiple code snippets retrieved as context, system instructions, and the issue description. The repeated and overlapping nature of this content across requests means a serving system that re-processes every prompt from scratch wastes a large fraction of compute on tokens it has already seen. This redundancy directly translates to high time-to-first-token (TTFT) and high cumulative cost per task. The latency cost is particularly important because agentic workflows are interactive: a user typically waits for the agent’s output, and slow generation degrades developer productivity in a way that aggregates across many sessions.

The standard solution to this redundancy is KV cache reuse, where the serving engine stores intermediate attention states from previous requests and reuses them when the same tokens appear again. Modern serving engines such as vLLM [Kwo+23] and SGLang [Zhe+24] implement prefix caching, which reuses cached KV states only when prompts share an exact prefix from the start. In an agentic setting, however, calls often share partial contexts that appear after some unique tokens, leading to low cache hit rates under prefix-only matching. CacheBlend [Yao+25] addresses this by enabling *position-independent* KV cache reuse: the prompt is treated as a sequence of chunks, each chunk is matched against the cache independently, and a small subset of cross-attention-

sensitive tokens are recomputed to restore correctness. The original CacheBlend paper demonstrates $2\text{--}3\times$ TTFT reduction and $3\text{--}5\times$ throughput improvement on multi-round question-answering workloads.

CacheBlend’s original evaluations, however, did not cover the workload characteristics of code agents. Agentic prompts are longer, often contain code blocks that are reordered or partially substituted across successive requests, and arrive at higher concurrency. Two specific properties of the reference implementation (LMCache) limit its applicability in this regime: cache lookup requires contiguous prefix alignment, so reusable chunks that follow a differing prefix are missed; and the storage backend allocates one memory object at a time under a global lock, which serializes large requests with many chunks. The per-step retrieve path is also opaque under existing instrumentation, making it difficult to diagnose where time is spent when speedup is lower than expected.

This thesis presents a set of enhancements to LMCache that address these limitations, together with an end-to-end evaluation under realistic code-agent workloads. The core technical contributions are on the backend serving system:

- **Non-contiguous cache hit support:** the segment token database is modified so that chunks following a masked prefix are hashed independently, allowing any subset of previously seen chunks to be reused regardless of their position in the current prompt.
- **Enhanced memory allocation and locking:** a batched varied-shape allocator reduces lock contention proportionally to the number of cache chunks per request, and a bounded eviction loop prevents deadlocks when all cache entries are pinned by concurrent lookups.
- **Per-request profiling infrastructure:** a thread-local profiler instruments the cache engine, storage backend, and GPU connector along the critical path, emitting structured per-operation and per-phase timing records for post-hoc analysis.

To evaluate these enhancements under realistic agentic traffic, the system is wrapped in an end-to-end pipeline that includes a context retrieval frontend. Producing a realistic agentic workload requires a method for assembling the per-request prompts that a real code agent would issue, including repository structure, relevant code snippets, and an issue description. We use RepoGraph [Ouy+25] combined with the Agentless procedural framework [Xia+24] as a concrete instantiation of the frontend. The frontend is treated as a pluggable component: any context retrieval module that produces structured prompts with explicit chunk boundaries can be substituted for it without affecting the backend design. Our frontend evaluation primarily serves to verify that the backend cache reuse does not degrade output quality and that the integrated system performs consistently across multiple LLM backends.

The remainder of this thesis is organized as follows. Chapter 2 provides background on KV cache reuse and CacheBlend, then on context retrieval and RepoGraph, and motivates an integrated system that supports realistic backend evaluation. Chapter 3 presents the system design, with emphasis on the backend cache engine enhancements that constitute our core contribution, followed by the frontend integration details required to complete the end-to-end pipeline. Chapter 4 reports the latency evaluation of the enhanced backend

under agentic workloads, followed by the supporting performance evaluation of the context retrieval frontend across multiple LLM backends.

Background

This chapter provides the technical background needed to motivate the system design in Chapter 3. Section 2.1 covers the latency bottleneck caused by redundant prefill computation in agentic workflows and presents CacheBlend as the state-of-the-art approach to position-independent KV cache reuse. Section 2.2 covers context retrieval as the complementary frontend concern and presents RepoGraph as the concrete retrieval module used in our integrated system. Section 2.3 explains why an integrated system is needed to evaluate the backend enhancements under realistic agentic workloads.

2.1 Position-Independent KV Cache Reuse

A primary source of latency in agentic LLM serving is redundant computation across requests. Many agentic frameworks send a large number of sequential requests to the LLM during a single task. The prompt in each request typically includes bulk context information and template tokens. When no cache reuse is in place, the model re-processes these tokens from scratch every time. For instance, an agent might retrieve a large snippet of code, feed it into the model to propose a fix, and then in a subsequent step feed much of the same snippet again while testing or refining the fix. Each of these calls recomputes attention over the repeated context tokens. This redundant computation is a shared inefficiency across many agentic workflows.

At the core of this inefficiency is the key-value (KV) cache in transformer models, which stores intermediate attention states of processed tokens so that each new token need not recompute attention over the entire sequence. The cache normally resets between separate API calls. To reuse computation across calls, several optimized LLM serving engines have introduced caching and scheduling strategies. vLLM [Kwo+23] improves throughput by batched execution and paged memory management of the KV cache, treating it as pageable memory. It can serve multiple generation requests in parallel and merge requests with identical prefixes so that the shared prefix is computed only once. Even vLLM, however, only reuses prefix computations when those prefixes exactly align from the start of the prompt. In an agent scenario, calls often share partial contexts that appear after some

unique tokens, leading to low cache hit rates. SGLang [Zhe+24] introduces RadixAttention, storing caches in a radix tree keyed by prefixes and scheduling requests to maximize cache hits, though it too relies on exact prefix matching. Another approach, Prompt Cache [Gim+24], allows explicitly marking reusable prompt segments and precomputing their KV states. While this yields substantial latency reductions on long-document QA tasks, it requires manual template setup and cannot adapt if templates change.

Cache optimization for LLM inference has recently taken a leap forward with CacheBlend [Yao+25]. CacheBlend’s key innovation is enabling *position-independent* reuse of KV caches. It treats the prompt as composed of chunks, checks for each chunk whether it has stored KV state, and when reassembling chunks selectively recomputes a small subset of tokens. The recomputation target is the set of HKVD (High-KV-Deviation) tokens, which are tokens whose cached KV state would otherwise deviate too far from what the model would have computed if the prompt were processed from scratch. Recomputing these tokens restores the effect of cross-attention on the forward-attention matrix. The observation that HKVD tokens tend to remain deviated through different layers also makes it possible to select and recompute them efficiently. This blending enables much higher cache hit rates on retrieval workflows, yielding 2–3 $\times$ reduction in time-to-first-token and 3–5 $\times$ throughput improvements without degrading output quality. CacheBlend was demonstrated effective in multi-round QA scenarios through datasets such as 2WikiMQA and Musique [Ho+20; Tri+22].

CacheBlend’s reference implementation, LMCache, leaves several aspects of agentic workloads unaddressed. First, cache lookup assumes contiguous prefix alignment: if any leading tokens are already cached by the serving engine (e.g., vLLM’s prefix cache), only the suffix following that prefix is eligible for segment-level cache matching. Agentic prompts, however, frequently share context chunks that appear after differing prefixes, and the original scheme misses these reusable suffixes. Second, the storage backend allocates one memory object per cache chunk, acquiring the storage lock on each allocation. For a prompt with N chunks this serializes N lock acquisitions, which becomes a bottleneck under high concurrency. Third, the eviction loop on a full cache retries indefinitely, which can deadlock when all entries are pinned by concurrent lookups on other threads. Finally, the per-step retrieve path is opaque under existing instrumentation, which makes it hard to attribute speedup or regression to specific phases of the cache engine. Chapter 3 presents the modifications introduced to address these four limitations.

2.2 Repository-Aware Context Retrieval

While cache reuse on the backend addresses latency, the quality of the output the agent produces depends on the context that is included in the prompt. This is a separate concern from the backend, but a complete end-to-end system requires some method for assembling per-request prompts that contain the right code context for a given task. We briefly review the state of context retrieval here because the frontend used in our evaluation builds on this prior work.

Modern code LLM agents have shown impressive capabilities, but most still strug-

gle with incomplete or irrelevant context when working on large repositories. Many frameworks, including SWE-Agent [Yan+24], AutoCodeRover [Zha+24c], and Agentless [Xia+24], feed the model only limited snippets of a codebase or generic summaries. Conventional agents typically interact with the entire codebase via searching keywords, files, and directories, plus some scrolling mechanism for file viewing. SWE-Agent’s implementation, for example, consists of three commands: `goto`, `scroll_up`, and `scroll_down`, which allow the agent to move a constant-sized context window to any line of the target file [Yan+24]. This linear view of the repository can leave the agent with false assumptions on code structure and implementation details, leading to unstable patch quality. For example, providing an agent with only a function implementation but not the definitions of its referenced symbols can omit crucial information and hurt the quality of patches. Conversely, naively supplying too much code can overwhelm the model with irrelevant details and interfere with its reasoning [Zha+24a; Zha+24b].

Several lines of work have addressed this tension. SWE-Search [Ant+25] integrates Monte Carlo Tree Search to explore the solution space, and CodePlan [Bai+23] tracks incremental code changes with dependency analysis. CoSIL [Jia+25] dynamically constructs a module-level call graph during the agent’s search, expanding the graph by following function calls and pruning irrelevant branches. RepoGraph [Ouy+25] takes a complementary approach by pre-building a complete repository graph before the agent starts working. It constructs a repository-level code graph that captures relationships between code elements across different files: each class, method, or function is a node, and the edges connect definitions to their usage or dependency. During a setup phase, RepoGraph parses all code files using `tree-sitter` to identify definitions and references, filters out non-code files, and stores the resulting graph for fast querying. When the agent needs context for a particular function or error, it queries this graph to retrieve all related definitions and usage chains. RepoGraph has been shown to consistently boost the resolve rate of Agentless and SWE-Agent when paired with capable backends such as GPT-4o or Claude-3.5-Sonnet [Ouy+25].

We use RepoGraph as the concrete context retrieval module in our integrated system. The choice is pragmatic rather than essential to the contribution: RepoGraph produces structured, chunk-friendly prompts that expose natural cache boundaries, and its open-source implementation is straightforward to integrate. The adapter design in Section 3.2.1 keeps the frontend interface narrow, so any alternative retrieval module that produces prompts with explicit chunk boundaries (for example a more recent agentic-search-based system) can be substituted without affecting the backend.

2.3 Motivation for Integration

The original CacheBlend evaluations used multi-round question-answering datasets in which prompts share structured text chunks across turns. While these workloads validate the position-independent reuse mechanism in principle, they leave open the question of how the system performs under code-agent workloads, where prompts can consist of long code context, repository structure information, and reordered context blocks across

requests. To evaluate this in a realistic setting, we wrap LMCache (with the backend enhancements introduced in Chapter 3) in an end-to-end agentic pipeline that uses RepoGraph and Agentless as the frontend to generate realistic prompts. The backend enhancements are the central technical contribution; the integration with the frontend serves as the evaluation harness that exposes the cache engine to representative agentic traffic.

A second motivation for the integrated system is a quality control point. Position-independent cache reuse trades a small amount of computed-from-scratch fidelity for substantial latency savings, and it is important to confirm that the resulting outputs remain usable in practice. Our frontend evaluation in Section 4.2 addresses this question directly by comparing the resolve rate of the integrated system under different blending granularities against a no-blending baseline.

The mechanism that ties the backend cache engine to the frontend prompts is what we refer to as *Cached Context*: prompts produced by the frontend are chunked at a per-context-object granularity, so that the backend can exploit reuse opportunities at the level of individual code snippets, repository structure blocks, or template segments. This is particularly effective in agentic settings because code-agent context is modular and repetitive: it consists mostly of repository structure information and function or class definition blocks, and prompt templates contain tokens that recur across every request. Chapter 3 formalizes this concept and presents the full system design.

System Design

This chapter presents the design of the end-to-end system. We first introduce the Cached Context concept that organizes prompts so that the backend cache engine can exploit reuse opportunities (Section 3.1). We then describe the system architecture, with emphasis on the backend serving path that constitutes our central contribution and a brief account of the frontend that feeds it (Section 3.2). Section 3.3 details the four backend enhancements that adapt CacheBlend’s reference implementation to agentic workloads. Section 3.4 covers the implementation details on the frontend side that are needed to complete the end-to-end pipeline.

3.1 Cached Context

The Cached Context concept is a simple but effective way to maximize KV cache reuse opportunities in agentic workflows. The key idea is to chunk each prompt at a per-context-object granularity, where a context object corresponds to a code snippet containing a symbol definition, a repository structure tree, or another semantically coherent unit of context.

To illustrate the advantage of Cached Context over traditional prefix caching, consider the following example. Suppose the agent constructs a prompt after context retrieval, consisting of four parts: system instructions, repository structure, issue description, and a block of relevant code snippets. A naive chunking strategy would treat this prompt as a single unit or divide it into four coarse blocks. With Cached Context, we further decompose the context blocks into finer-grained chunks. The relevant code snippets block, for example, might contain definitions for multiple symbols retrieved by RepoGraph, where each symbol definition is now its own cache chunk.

Now consider a subsequent prompt in the same workflow, but with slightly different localization and a partly different set of code snippets, while most symbols and other blocks remain the same. Under prefix caching, unless the overlapping content appears from the start of the prompt, the cache cannot be reused. Under Cached Context, each overlapping symbol definition is a cache hit regardless of its position. This position-independent reuse

is particularly valuable in agentic workflows, where the order and combination of context elements frequently varies.

3.2 System Architecture

Figure 3.1 illustrates the overall architecture. The design prioritizes modularity, allowing each component to be developed and tested independently. The remainder of this section walks through the backend and frontend in that order.

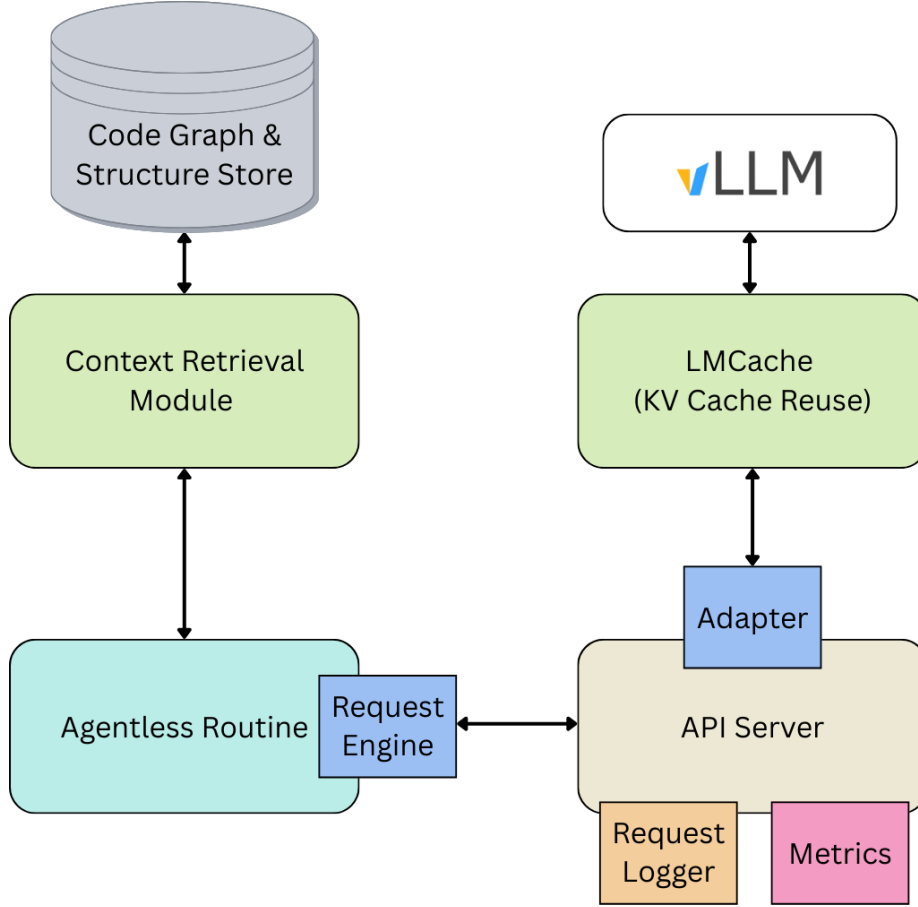


Figure 3.1: End-to-end system architecture.

3.2.1 Backend: API Server with CacheBlend

The backend consists of an API server that serves as the entry point for frontend requests. The server hosts the LLM inference engine (vLLM) with LMCache integration, which implements the CacheBlend algorithm for position-independent KV cache reuse. When a request arrives, the server performs the following steps:

1. Parse the chat messages from the request.

2. Pass the messages through the adapter module (described below) for tokenization and chunk extraction.
3. Forward the processed prompt to the LLM engine with CacheBlend enabled.
4. Return the generated response to the client.

The backend exposes a chat-completion interface compatible with the OpenAI API format. This means any compatible frontend can drive the backend without modification beyond annotating context-object boundaries in its prompt templates.

3.2.2 Adapter Module

The adapter module serves as the bridge between the frontend prompt format and the backend cache engine. Its responsibilities include:

1. **Chunk extraction:** identifying and retrieving individual context objects from the incoming chat messages.
2. **Tokenization:** encoding each chunk using the model’s tokenizer, including correct handling of models with custom tokenization schemes (e.g., Devstral).
3. **Chunk stitching:** reassembling the tokenized chunks to produce the final prompt token sequence consumed by the LLM engine.

The adapter’s design supports flexible chunking granularity based on the frontend prompt template and user preference. If chunking is per symbol, the frontend adds a delimiter after each code snippet fetched during code graph exploration. If a coarser granularity is desired (e.g., per context block), delimiters can be placed only between major sections of the prompt. This flexibility makes the system robust to various workload patterns and to differing optimal chunk sizes.

3.2.3 Frontend: RepoGraph + Agentless

We use RepoGraph as the frontend context retrieval algorithm, integrated with the Agentless procedural framework. Agentless follows a three-step routine:

1. **Localize:** given an issue description, identify the files and code regions most likely to require modification.
2. **Repair:** generate candidate patches for the localized regions.
3. **Rerank:** evaluate and rank the candidate patches, selecting the most promising one for submission.

RepoGraph enhances the localization phase by providing repository-aware context. Rather than relying solely on file-level search, Agentless can query RepoGraph’s code graph to retrieve symbol definitions, call chains, and structural information. This integration ensures that the LLM receives semantically complete context. For example, when localizing a bug in a function, the model also receives definitions of the symbols referenced by that function.

To enable Cached Context, we modify Agentless’s prompt generation routine to make context object boundaries explicit, which exposes chunk boundaries to the backend adapter

without affecting the semantic content of the prompt. The frontend is otherwise treated as a pluggable component: any retrieval module that produces prompts with explicit chunk boundaries can replace RepoGraph + Agentless without changes to the backend.

3.3 Backend Enhancements

This section presents the four backend enhancements that adapt CacheBlend’s reference implementation (LMCache) to the workload characteristics of code agents. Each enhancement targets a specific limitation identified in Section 2.1: non-contiguous chunk reuse, lock contention and deadlocks in memory allocation, and the lack of fine-grained diagnostic instrumentation. A fourth subsystem (request logging and replay) is also part of the backend because it underpins the latency evaluation methodology in Chapter 4.

3.3.1 Non-Contiguous Cache Hit Support

The original CacheBlend implementation requires contiguous prefix alignment for cache lookup. If the prompt begins with tokens already cached by the serving engine (e.g., vLLM’s prefix cache), only the suffix following that prefix is eligible for segment-level cache matching. In an agentic workflow, however, the set of context objects retrieved across successive requests frequently overlaps only partially and in non-consecutive positions. For example, if prompt A includes context chunks $\{C_1, C_3, C_4\}$ and a subsequent prompt B includes $\{C_2, C_3, C_4\}$, the original scheme would miss the reusable suffix $\{C_3, C_4\}$ because C_2 differs from C_1 and breaks prefix contiguity.

We modify the segment token database to hash each chunk independently after the masked prefix. Given a mask of the form $F \dots FT \dots T$, where leading F entries denote tokens already handled by the engine’s prefix cache, each segment among the remaining T entries is hashed and looked up in the cache store separately. This enables non-contiguous cache hits: any subset of chunks that appeared in a prior request can be reused regardless of its position in the current prompt. The modification preserves backward compatibility: prompts with contiguous overlap behave identically to before. In multi-turn agentic workloads where context objects are reordered or partially substituted between requests, this change substantially increases hit rates.

3.3.2 Enhanced Memory Allocation and Locking

Scaling CacheBlend to agentic workloads with high request concurrency and large prompt lengths exposed two bottlenecks in the storage backend’s memory management path.

Batched varied-shape allocation. Each cache chunk corresponds to a memory object whose shape depends on the chunk’s token count. The original implementation allocated these objects one at a time, acquiring and releasing the CPU storage lock for each chunk. Under high concurrency, this per-chunk locking becomes a serialization bottleneck: a single store operation for a prompt comprising N chunks requires N lock acquisitions. We

introduce a `batched_allocate_varied` method that accepts a list of shapes in one call, acquires the lock once, performs all allocations (with bulk eviction if capacity is exceeded), and releases the lock. This reduces lock contention by a factor of N per request and avoids partial-allocation states where some chunks succeed while others fail due to concurrent eviction.

Bounded eviction with deadlock prevention. When the cache is full, the allocator must evict existing entries to make room. In the original design, the eviction loop retried indefinitely until sufficient space was freed. Under heavy concurrent load, this could deadlock: if all cache entries were pinned by in-flight lookups on other threads, no entry was eligible for eviction, and the allocator would spin indefinitely while holding the storage lock. We bound the eviction loop to a configurable maximum number of retries. If the limit is reached, the allocation fails gracefully and the request falls back to full prefill computation, avoiding system-wide stalls.

3.3.3 Per-Request Profiling Infrastructure

To enable fine-grained latency analysis of the cache engine under realistic workloads, we implement a per-request profiling module that instruments the critical path through the adapter, cache engine, storage backend, and GPU connector. The profiler is organized around two abstractions: *operations*, which correspond to logical units of work (e.g., a single request’s retrieve pass), and *phases*, which subdivide an operation into timed intervals (e.g., token database lookup, per-layer storage retrieval, per-layer GPU onload).

The profiler uses thread-local storage to maintain a per-thread operation stack, ensuring that timing records from tensor-parallel workers do not bleed across threads. Each phase is recorded with its wall-clock duration, parent operation identifier, and thread ID. On completion, the profiler emits per-operation summaries in JSONL format, enabling post-hoc aggregation and visualization.

Figure 3.2 shows an example timeline produced by the profiler during a CacheBlend-enabled serving session. Each point represents a single profiled operation, plotted by wall-clock time on the x-axis and measured latency on the y-axis, and color-coded by operation type. The `retrieve_layer` operations (orange) constitute the bulk of per-step cache engine overhead, typically clustering between 25 and 75 ms with occasional spikes above 100 ms. The `store_layer` operations (green) show a similar range, corresponding to the cost of persisting newly computed KV states. The `adapter.start_load_kv` markers (red, near the baseline) indicate the lightweight entry point into the cache path. This visualization enables rapid identification of latency outliers and temporal patterns that would be invisible in aggregate statistics.

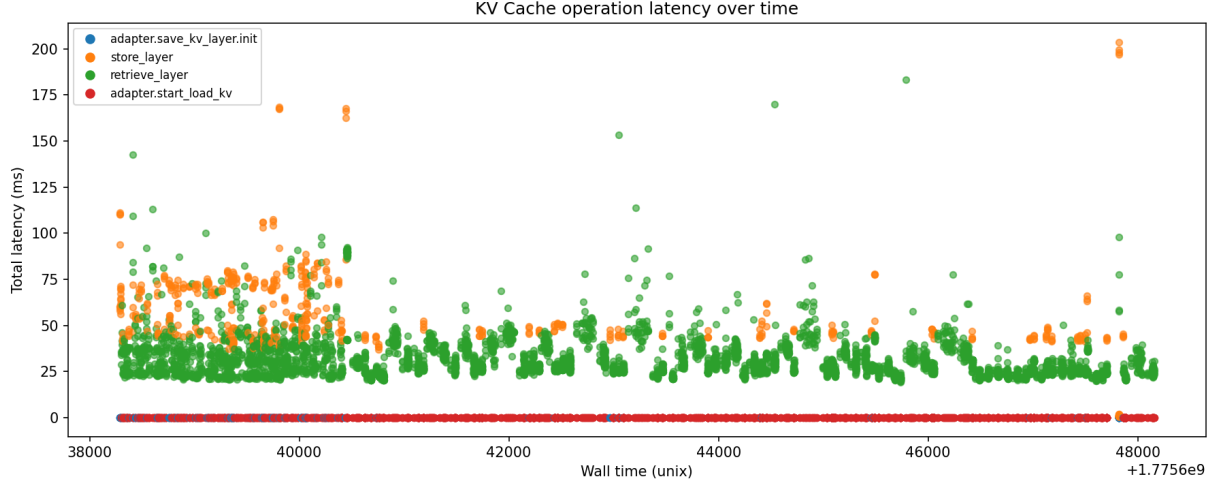


Figure 3.2: Example profiler output: per-operation latency timeline during a CacheBlend-enabled serving session (chunk lower bound 128 tokens). Each point is a single profiled operation, color-coded by type. The clustering of `retrieve_layer` (orange) at 25–75 ms reflects the fixed per-step overhead analyzed in Section 4.1.3.

This infrastructure is critical for diagnosing performance anomalies that are invisible at the aggregate level. As discussed in Section 4.1.3, the profiler’s per-request phase breakdown revealed the root cause of high speedup variance among small requests, an insight that would not have been possible from end-to-end TTFT measurements alone.

3.3.4 Request Logging and Replay

To support debugging, profiling, and reproducible experimentation, the API server includes a request logging module. During a server session, all incoming requests and their corresponding responses are captured and stored as a local trace file.

This transparency enables inspection of which stage during the Agentless routine may have caused a performance bottleneck. For example, if localization prompts consistently exhibit low cache hit rates, the prompt template can be adjusted to improve context alignment across requests.

In addition, the server supports a **replay** functionality that parses requests from any trace file and simulates the workload internally. This feature enables:

- Evaluation on captured traces from real agentic workloads without re-running the full agent pipeline.
- Modularized testing through synthetic small traces that exercise specific caching scenarios.
- A/B comparison of different caching configurations on identical workloads.

These capabilities are essential for the latency evaluation methodology in Chapter 4. The replay mechanism lets us drive the backend with a fixed, deterministic trace under both blend and no-blend configurations, eliminating the variability that would otherwise come from re-running the agent loop end-to-end.

3.4 Frontend Integration Details

Beyond the core architecture, several implementation details on the frontend side improve the system’s robustness and the realism of the prompts that reach the backend.

3.4.1 Context Truncation

For complex coding tasks involving multiple files and nested dependencies, the frontend can exceed the model’s token limit when constructing a prompt. Originally, prompts were naively truncated at the end when the token limit was reached. This approach frequently results in important instructions (such as output format requirements or validation criteria) being omitted from the prompt. Even when the LLM receives sufficient context to understand the problem, it may underperform due to missing instructions.

Our solution leverages the model’s tokenizer to calculate the token length of each context block and the prompt template. Before constructing the prompt, the last context block is truncated to exactly fit the remaining token budget, so that the critical pieces on the template stay intact and instructions are passed to the LLM as desired. This improves the reliability of the generated patches when the context is long.

3.4.2 Local Store for Precomputed Data

Computing the code graph and repository structure from scratch for every task instance incurs significant latency overhead. RepoGraph’s graph construction involves parsing all source files using `tree-sitter`, identifying symbol definitions and references, and building the dependency graph. For large repositories, this process can take several minutes.

To address this, we introduce a local store that caches these large data structures. The store is keyed by repository ID and commit hash, ensuring that cached data remain valid across tasks targeting the same codebase state. This optimization is particularly valuable when processing multiple issues against the same repository (common in benchmark evaluations like SWE-bench) or when the same repository is revisited across different agent sessions.

Evaluations

The evaluation comprises two parts. We first measure the latency benefits of the enhanced CacheBlend backend under agentic workloads, which is the primary evaluation that validates the central contribution of this thesis (Section 4.1). We then evaluate the frontend context retrieval algorithm across multiple LLM backends and verify that the backend cache reuse does not degrade output quality (Section 4.2).

4.1 Latency Evaluation for KV Reuse

To assess the latency benefits of CacheBlend under agentic workloads, we measure the time-to-first-token (TTFT) speedup achieved by position-independent KV cache reuse on traces generated by the RepoGraph frontend.

4.1.1 Setup

We evaluate the latency characteristics of CacheBlend by replaying agentic traces generated by the RepoGraph + Agentless frontend against vLLM with and without LMCache integration. All experiments use Devstral Small served with 4-way tensor parallelism. For the blend condition, LMCache is integrated with CacheBlend enabled at a minimum chunk size of 128 tokens. For the baseline condition, the same model is served without LMCache, using only vLLM’s native prefix caching. Both conditions process the identical trace via the replay mechanism described in Section 3.3.4.

Metric

We adopt TTFT as our primary latency metric, defined as the elapsed time from request arrival to the emission of the first output token. TTFT directly captures the prefill-phase cost, which is the phase that cache reuse is designed to accelerate. For each request, we compute the *TTFT speedup* as the ratio of baseline TTFT (no blending, no LMCache) to blend-mode TTFT. A speedup greater than 1.0 indicates that cache reuse reduced prefill latency; a value below 1.0 indicates overhead-induced regression.

Datasets

We replay the full set of prompts produced by RepoGraph + Agentless on SWE-bench Verified-mini [Mar25]. The resulting trace comprises 537 requests spanning a wide range of prompt lengths (119 to over 100,000 tokens) and cache hit ratios (0% to 99.2%), providing coverage across the operating regimes most relevant to agentic use.

Model

All experiments use Devstral Small, which is also the open-source model used in the performance evaluation (Section 4.2). Using a single model in both evaluations isolates the effect of the backend cache reuse from any model-to-model variation.

Configurations

We compare the following two configurations:

- **Baseline** (vLLM with prefix caching): vLLM with its built-in prefix caching enabled, which reuses KV states only for exact prefix matches. No LMCatch integration.
- **Experimental** (vLLM + LMCatch with CacheBlend): vLLM with LMCatch integration and CacheBlend enabled, allowing position-independent KV cache reuse across non-contiguous segments.

4.1.2 Results

Speedup as a Function of Cache Hit Ratio and Prompt Length

Figure 4.1 presents the central result of our latency evaluation: a 3D surface plot of TTFT speedup as a function of cache hit ratio and total prompt tokens, with IQR-based outlier removal ($3.0\times$ threshold) applied to remove anomalous data points. The fitted plane

$$\text{speedup} = 1.479 \times \text{hit_ratio} + 1.09 \times 10^{-6} \times \text{total_tokens} + 0.427$$

achieves $R^2 = 0.610$ ($n = 532$), indicating that cache hit ratio is the dominant predictor of speedup while total token count has negligible independent effect. The breakeven point, where CacheBlend’s overhead is exactly offset by the compute savings from cache reuse, sits at approximately 39% cache hit ratio.

3D: Speedup vs Hit Ratio vs Total Tokens

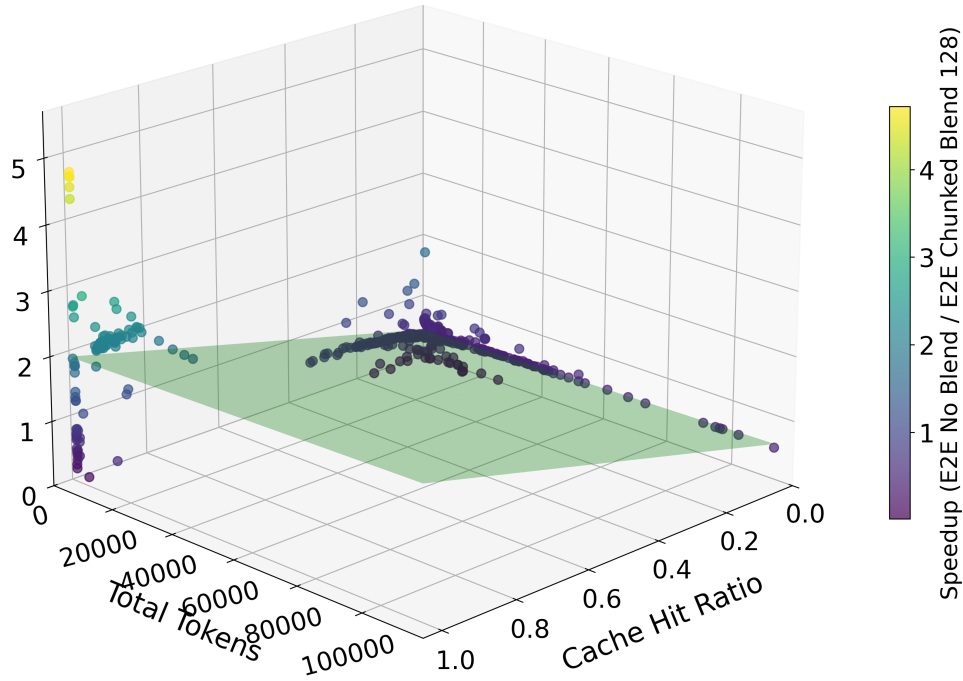


Figure 4.1: 3D surface plot of TTFT speedup vs. cache hit ratio and total prompt tokens (outliers removed). The fitted plane shows that hit ratio is the primary driver of speedup, with a breakeven point at $\sim 39\%$ hit ratio.

Two distinct behavioral regimes are visible in the plot. For requests with more than 10,000 tokens, nearly all data points cluster tightly around the fitted surface, indicating that CacheBlend’s speedup is highly predictable for long-context workloads. In contrast, requests below 5,000 tokens exhibit a pronounced fan-shaped spread: some achieve up to $4\text{--}5\times$ speedup while others regress below $0.5\times$, despite comparable hit ratios.

Figure 4.2 breaks down the speedup distribution by token-count bucket. The $<2k$ bucket has a mean speedup of 0.862 but an absolute range of 0.036 to 9.239, confirming that short-prompt behavior is highly unpredictable. The 5–10k bucket shows a tighter distribution centered near breakeven (mean 0.996, with 33% of requests above $1.0\times$), and the $>30k$ bucket is consistently above breakeven. This pattern further supports the observation from the 3D plot: CacheBlend’s latency benefit is most reliable for the long-context prompts that are also the most expensive to recompute from scratch.

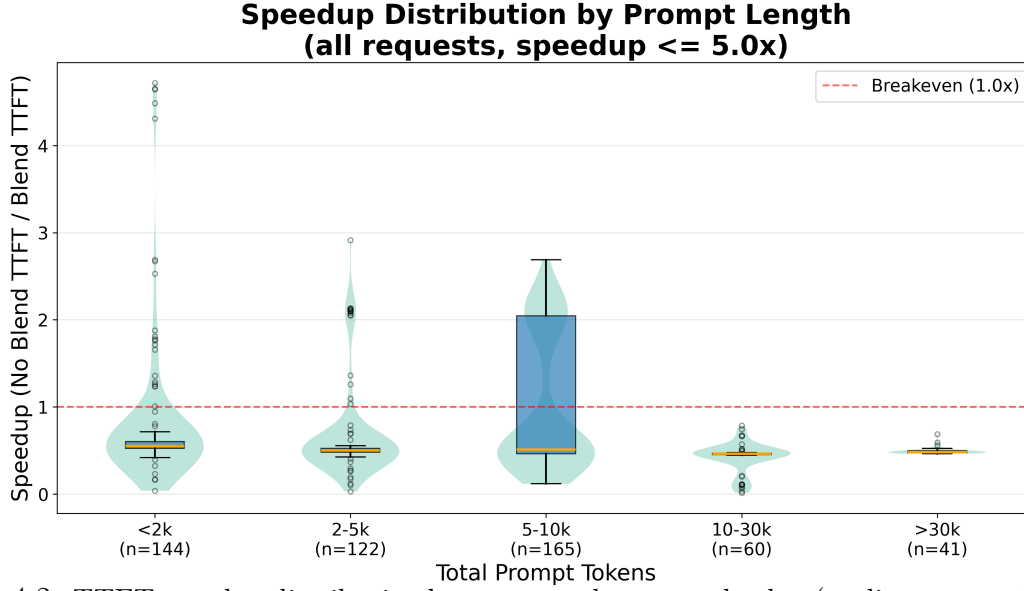


Figure 4.2: TTFT speedup distribution by prompt token-count bucket (outliers removed). Violin plots show the probability density; box plots mark the median and interquartile range.

Notably, this variance among small requests persists even when restricting to well-cached requests. Among requests with cache hit ratio at least 50%, the $<2k$ bucket retains a spread from $0.3\times$ to $9.5\times$, while the $5\text{--}10k$ bucket collapses to a tight $1.9\text{--}2.5\times$ band. A high cache hit ratio alone does not guarantee fast performance for small requests; additional factors govern the outcome. We investigate these factors in detail in Section 4.1.3.

Speedup Distribution

Figure 4.3 shows the distribution of per-request TTFT speedup values across the full trace. The distribution is bimodal: the primary mode lies below $1.0\times$ (median $0.51\times$, mean $0.77\times$), reflecting the overhead-dominated short-request regime discussed in Section 4.1.3. A secondary mode near $2.0\times$ corresponds to longer-prompt requests where cache reuse outweighs the per-step overhead. While the majority of individual requests experience a net slowdown under the current implementation, the high-speedup requests tend to be the most compute-intensive ones (exceeding 10,000 tokens), where absolute time savings are largest. It is worth noting that the SWE-bench Verified-mini trace is derived from real-world Django issues, which tend to produce shorter, more localized prompts compared to repositories with deeper dependency chains. As a result, the request length distribution skews toward the short-prompt regime where per-step overhead dominates. Workloads targeting larger or more interconnected codebases would shift the distribution toward longer prompts and higher cache hit ratios, where CacheBlend’s speedup is both more consistent and more substantial.

Speedup Distribution (E2E No Blend / E2E Chunked Blend 128)

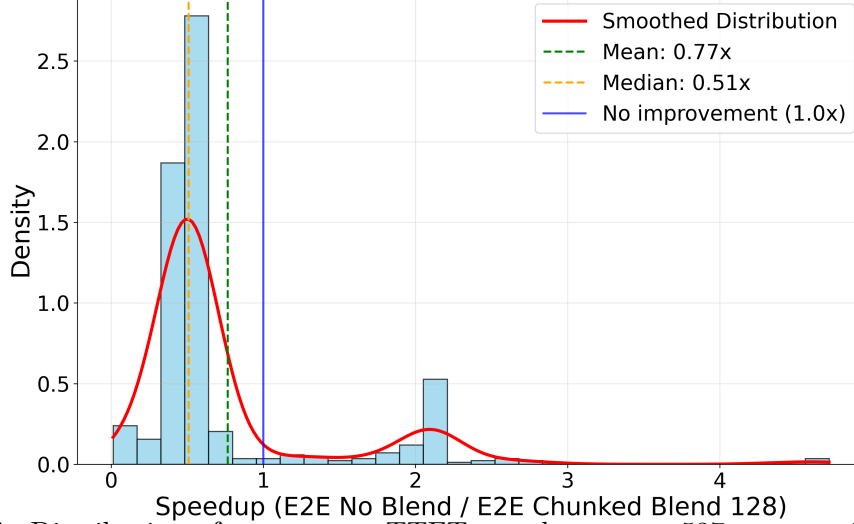


Figure 4.3: Distribution of per-request TTFT speedup across 537 requests in the SWE-bench Verified-mini trace.

4.1.3 Analysis: Small-Request Speedup Variance

The 3D speedup plot reveals that requests with fewer than 5,000 tokens exhibit significantly higher speedup variance than longer requests at comparable hit ratios. Using the per-request profiling infrastructure described in Section 3.3.3, we trace the root cause to an interaction between vLLM’s chunked-prefill scheduler and LMCache’s per-step invocation pattern.

Scheduler-Induced Fragmentation

vLLM’s chunked-prefill scheduler determines how many tokens to process per scheduling step based on `max_num_batched_tokens` and the current batch composition. A request that arrives during a decode-heavy window is interleaved across multiple steps, each processing only a fraction of the prompt. The profiler records reveal a bimodal distribution in the number of `retrieve_layer` operations per logical request: 322 requests incur 4 operations (1 prefill step $\times$ 4 tensor-parallel workers) while 99 requests incur 36 operations (9 prefill steps $\times$ 4 workers).

Figure 4.4 cross-tabulates the fragmentation rate against prompt length. Small requests ($<2,000$ tokens) are fragmented at a 45% rate, which is $3.4\times$ the rate of 5–10k requests (10%) and the highest of any bucket. This susceptibility arises because short prompts fill a smaller portion of `max_num_batched_tokens`, leaving room for the scheduler to pack decode steps alongside them and defer their remaining tokens to subsequent batches.

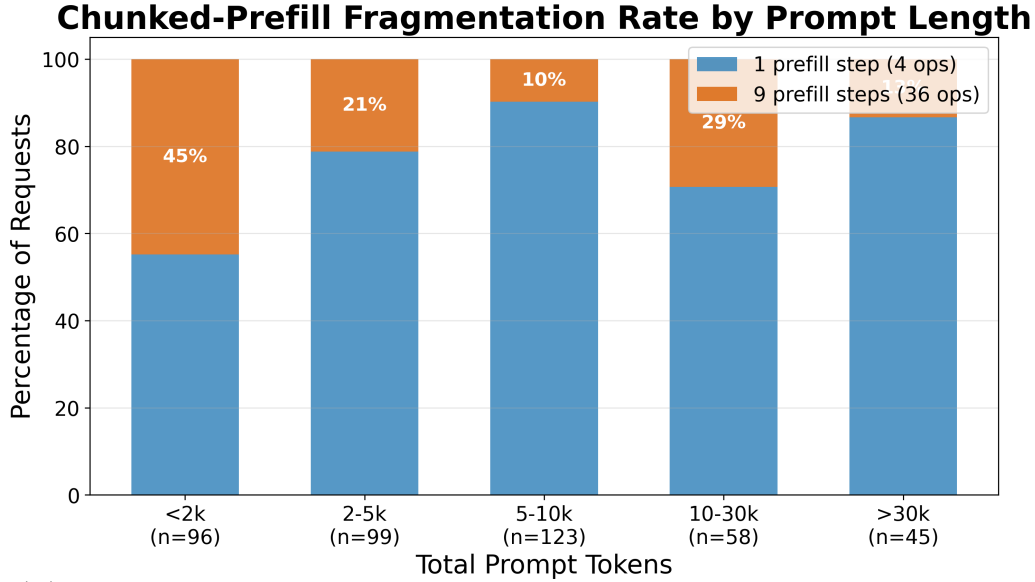


Figure 4.4: Chunked-prefill fragmentation rate by prompt token-count bucket. Requests are classified as non-fragmented (1 prefill step, 4 ops) or fragmented (9 prefill steps, 36 ops). Small requests (<2k tokens) exhibit the highest fragmentation rate at 45%.

Fixed Per-Step Overhead

Each invocation of the cache engine’s retrieve path triggers a chain of operations whose cost is largely independent of the number of tokens in the step. Figure 4.5 presents the measured per-call cost broken down by phase group, aggregated over 4,852 blend and 3,092 no-blend `retrieve_layer` calls. Three phases dominate:

1. **Token database lookup** (0.7 ms blend, 0.3 ms no-blend): walks the token sequence to identify cached chunks.
2. **Storage retrieval** (2.0 ms blend, 3.1 ms no-blend): fetches each layer’s KV cache from the CPU storage backend.
3. **GPU onload** (28.1 ms blend, 14.6 ms no-blend): copies KV data into vLLM’s paged buffer and, in blend mode, executes gap-aware recomputation.

GPU onload accounts for 91% of blend’s per-call time and exhibits a $1.93\times$ ratio over no-blend. The ~ 13.5 ms gap stems from CacheBlend’s blender, which selects the top 15% most-divergent token positions per layer and recomputes their KV state to restore cross-attention accuracy. The overall mean per-call overhead is 30.8 ms in blend mode versus 18.0 ms without blending (a $1.72\times$ ratio).

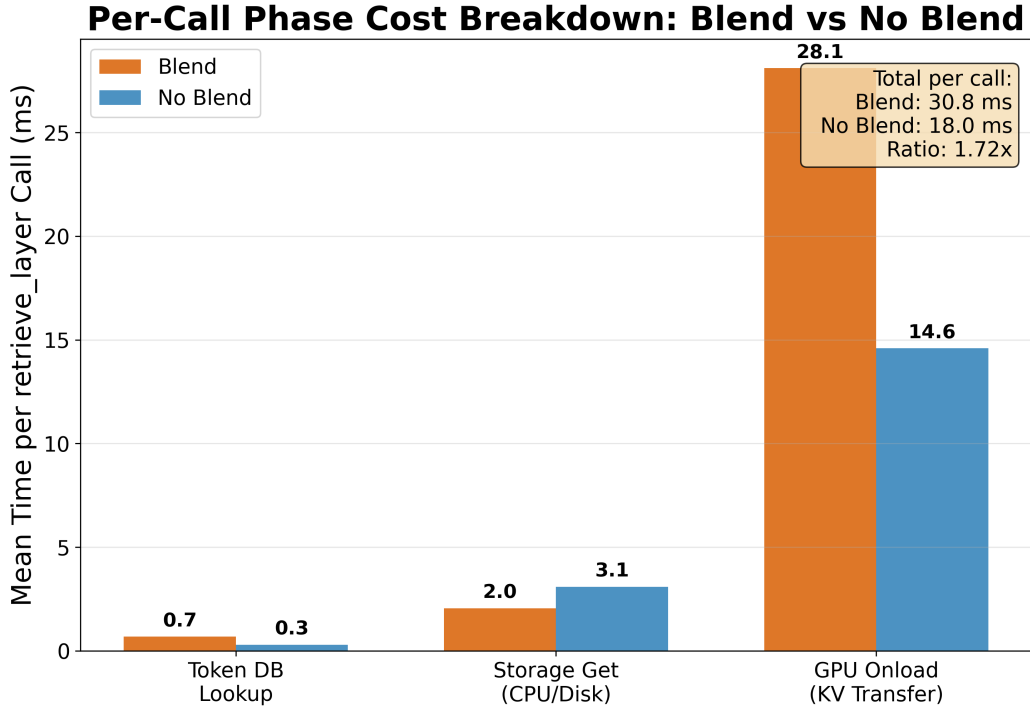


Figure 4.5: Per-call `retrieve_layer` cost broken down by phase group (blend vs. no-blend). GPU Onload dominates at 91% of blend’s per-call time, with a $1.93\times$ ratio over no-blend due to gap-aware KV recomputation.

The fixed-cost nature of this overhead creates a fundamental asymmetry with respect to prompt length. For a 1,000-token request fragmented across 9 steps with 4 tensor-parallel workers, the aggregate retrieve overhead reaches $\sim 1,109$ ms, which is $9.7\times$ the ~ 114 ms baseline TTFT for straight prefill of the same prompt. The compute savings from cache hits (~ 100 ms) are dwarfed by this overhead. By contrast, a 50,000-token request incurs the same $\sim 1,109$ ms of overhead, but against a $\sim 5,000$ ms baseline (22% of total), and cache hits save $\sim 4,500$ ms of compute, yielding a net $\sim 2\times$ speedup that closely matches the fitted surface. The per-step overhead is invariant to the number of tokens in the step, but the compute savings scale linearly with token count, so small requests sit below the crossover where overhead exceeds savings.

Fragmentation as the Source of Variance

The preceding analysis explains why small requests are slower *on average*, but does not fully account for the extreme *variance* observed in the $<5k$ token regime. Figure 4.6 resolves this by plotting speedup against total tokens on a log scale, with each point colored by its fragmentation class: non-fragmented (1 prefill step, 4 ops) versus fragmented (9 prefill steps, 36 ops).

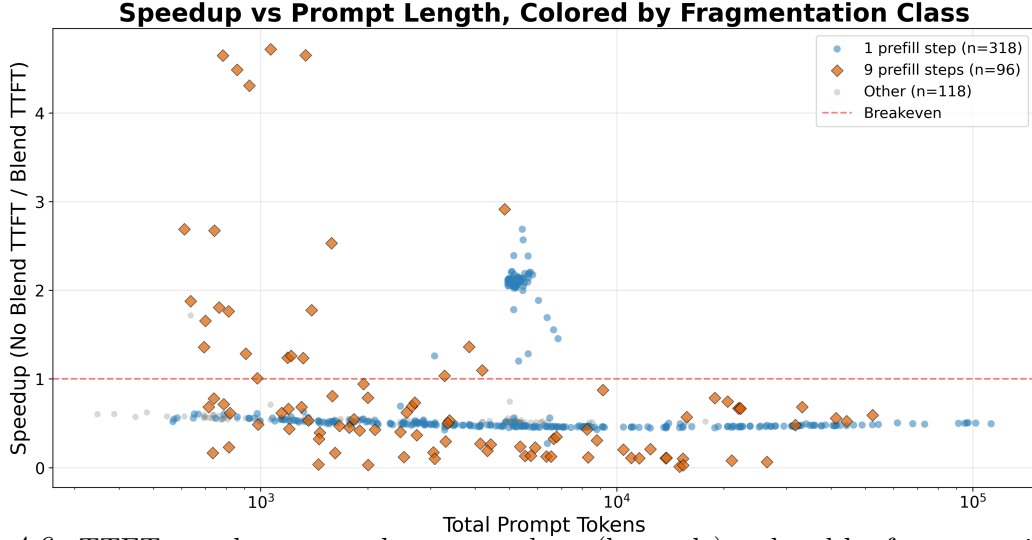


Figure 4.6: TTFT speedup vs. total prompt tokens (log scale), colored by fragmentation class. Non-fragmented requests (blue circles) cluster tightly below breakeven for small prompts; fragmented requests (orange diamonds) exhibit the full variance spread. Both classes converge for long prompts.

Two distinct populations emerge among small requests:

1. **Non-fragmented small requests** (blue, 4-op) cluster tightly at $\sim 0.54\times$ speedup for the $<2k$ bucket (mean 0.538, std 0.031). Blend overhead is consistent and always exceeds the compute savings for these short prompts. This population is predictable but consistently below breakeven: each request pays a single step of 30.8 ms overhead against a baseline TTFT that is itself only $\sim 60\text{--}200$ ms.
2. **Fragmented small requests** (orange, 36-op) exhibit the full spread from near $0\times$ to $9.5\times$ for the $<2k$ bucket (mean 1.604, std 1.786). The variance is driven by the interaction between the number of scheduler steps and the cache hit pattern across those steps. Some fragmented requests encounter favorable cache timing where cached content is warm from earlier steps in the same scheduling window, while others pay the full per-step overhead with minimal cache benefit.

The key insight is that **fragmentation is both the source of the worst under-performers and the best over-performers** among small requests. The scheduler’s batching decision at request arrival time determines which fragmentation class a request falls into, and this decision depends on the instantaneous batch composition rather than on the request’s own cache state. Two identically sized, identically cached requests can land on opposite sides of the speedup distribution based solely on whether other decode requests happened to be in flight at the moment of arrival.

For large requests ($>10k$ tokens), both fragmentation classes converge to the same narrow band around the fitted surface, because the per-step overhead is a small fraction of per-step compute regardless of how many steps the scheduler chose. This convergence confirms that the variance phenomenon is specific to the short-prompt regime and does not affect the primary target workload of long-context agentic prompts.

Implications

The fragmentation effect is not a fundamental limitation of CacheBlend’s reuse model but rather a consequence of how the per-step invocation pattern interacts with the scheduler’s batching decisions. Three mitigation strategies are under consideration:

- (a) A *minimum token-per-step threshold* that skips cache retrieval for micro-batched steps, directly addressing the pathological case at the adapter level. This is the lowest-risk fix: it can be implemented in the adapter’s per-request loop and tuned via the per-call overhead profile (30.8 ms blend, 18.0 ms no-blend).
- (b) A *prompt-length-aware bypass* that routes sub-2,000-token requests away from the blend path entirely. The data supports this approach strongly: even non-fragmented small requests average only $0.54\times$ speedup, indicating that blend is net-negative for short prompts regardless of fragmentation. Sub-2k requests represent $\sim 27\%$ of the trace population (145/536), and their absolute compute savings are modest (~ 100 ms).
- (c) A *coalesced retrieval mode* that accumulates token ranges across prefill steps and issues a single batched lookup, eliminating the overhead multiplier. This is the most principled long-term solution but requires a non-trivial redesign of the adapter–cache-engine interface, since the current layerwise pipeline interleaves layer loads with the model’s forward pass.

For deployments where the request length distribution is known, as in the SWE-bench trace used here, approaches (a) and (b) provide effective, low-complexity mitigations. Approach (b) is particularly appealing given the finding that even the non-fragmented population does not benefit from blend at short prompt lengths.

Having established the latency characteristics of the enhanced backend, we next turn to the frontend context retrieval evaluation, which serves both as a robustness check across LLM backends and as a quality control for the backend cache reuse.

4.2 Performance Evaluation for Context Retrieval

This section evaluates the frontend context retrieval algorithm (RepoGraph + Agentless) along two dimensions. First, we measure the resolve rate of the integrated agent across several LLM backends of different parameter sizes and architectures, to verify that the frontend behaves consistently when paired with different models. Second, we measure the resolve rate of the integrated agent under different CacheBlend blending granularities to verify that the backend cache reuse does not degrade output quality. Both evaluations serve as supporting evidence for the central latency contribution presented in Section 4.1.

4.2.1 Setup

The evaluation of RepoGraph largely follows the methods of the original paper, where it is utilized as a plug-in component on top of an existing agentic framework. For the purpose of our study, we chose the Agentless procedural framework. The advantage of integrating a procedural framework in lieu of a traditional agent is simplicity and comprehensibility. Agentless deploys a three-phase process of localization, repair, and patch validation [Xia+24]. Intermediate results are transparent between each step, where empty or error responses are immediately identifiable. This allows for finding bottlenecks and comparing performance across various models at step granularity.

Our goal is to verify the consistency of the integrated agent’s performance across LLM backends of different parameter sizes and architectures, and to verify that the backend cache reuse does not degrade output quality. For this purpose, we conducted experiments for RepoGraph + Agentless using a set of models with different configurations (for details see Models).

Metric

We adopt resolve rate following the original work as our primary evaluation metric. An instance is considered resolved if (1) the generated patch is successfully applied to the repository and (2) the patched code passes all test suites.

This metric provides a direct measure of end-to-end task success, capturing not only whether a patch is syntactically correct but also whether it integrates cleanly into the repository and satisfies the intended functionality. Compared to intermediate measures such as code similarity or partial compilation success, the resolve rate reflects the practical utility of the generated patches and therefore serves as a good indicator of real-world performance.

Datasets

For a comprehensive yet time- and cost-efficient evaluation, we run our experiments on SWE-bench Verified-mini [Mar25], which is a subset of SWE-bench Verified [Cho+24] selected via k-means clustering and linear programming so that the marginal distributions of important scores are preserved. This is a smaller dataset than SWE-bench Lite [Yan+24],

yet high-quality and well-specified, and is capable of reflecting an agent’s performance across a spectrum of task difficulties.

Models

To ensure consistency, robustness, and portability of RepoGraph as the repository-aware context retrieval algorithm, it is important to evaluate it against multiple model backends of varying architectures and parameter scales. Different models exhibit distinct behaviors in terms of tokenization, embedding spaces, and context utilization. A retrieval method that aligns well with one tokenizer or attention mechanism may underperform when paired with another, which makes cross-model testing critical for avoiding over-specialization.

In addition, models differ in how they exploit the retrieved context. More powerful models may compensate for imperfect retrieval given stronger reasoning and synthesis capabilities, whereas smaller models are more sensitive to the accuracy and relevance of the retrieved context. Evaluating across a spectrum of models provides insight into how much of RepoGraph’s performance benefits stem from the context retrieval strategy itself. This also allows us to characterize situations where retrieval quality is most impactful, particularly in cost-constrained settings where smaller models are attractive.

Finally, cross-backend evaluation provides practical benefits for deployment. Since real-world systems often migrate between providers (e.g., OpenAI, Anthropic, Mistral), retrieval algorithms must remain portable to ensure stable performance under backend substitution. For our experiments, we therefore consider a representative set of both proprietary and open-source models, including **GPT-4o**, **o4-mini**, **Claude 3.5 Sonnet**, and **Devstral Small**, covering a range of architectures, parameter counts, and costs.

4.2.2 Results

Figure 4.7 shows the resolve rate distribution for RepoGraph + Agentless across the four model backends on SWE-bench Verified-mini. The integrated agent achieves comparable resolve rates across the proprietary and open-source models considered, which supports the claim that RepoGraph generalizes across backends rather than being tuned to a specific model family. The Devstral Small results are particularly relevant for the integrated system evaluation, since Devstral Small is the open-source model used as the LLM backend in the latency evaluation (Section 4.1).

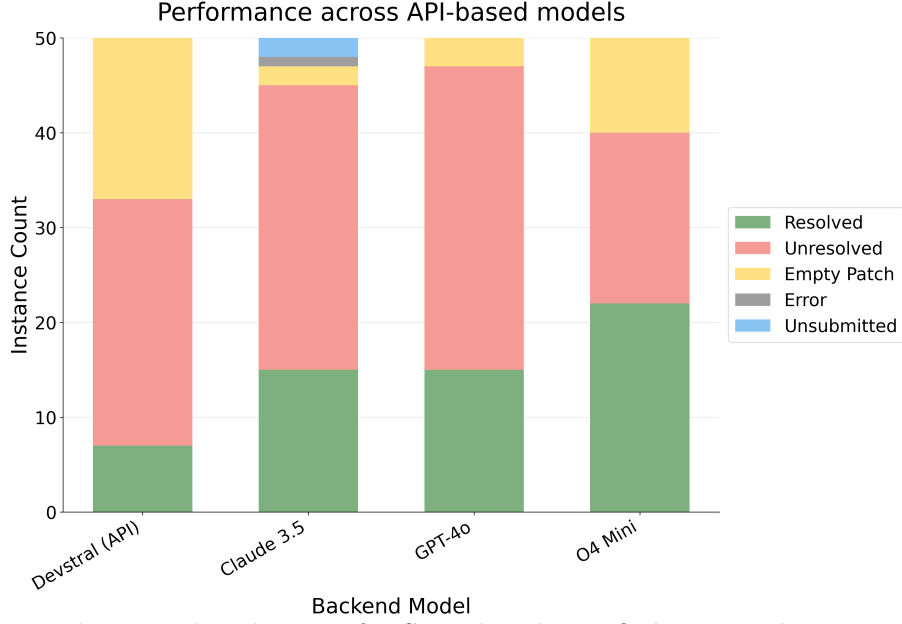


Figure 4.7: Resolve rate distributions for SWE-bench Verified-mini evaluation runs across different models.

To verify that CacheBlend’s KV cache reuse does not affect output quality, we run an additional evaluation using Devstral Small hosted on vLLM with LMCatche (CacheBlend enabled) on the same SWE-bench Verified-mini dataset. We compare different blending granularities against the baseline (no blending) to verify that position-independent cache reuse does not degrade patch quality. Figure 4.8 shows that the resolve rates across blending granularities are comparable to the no-blending baseline, confirming that the backend cache reuse preserves output quality even at finer chunk granularities.

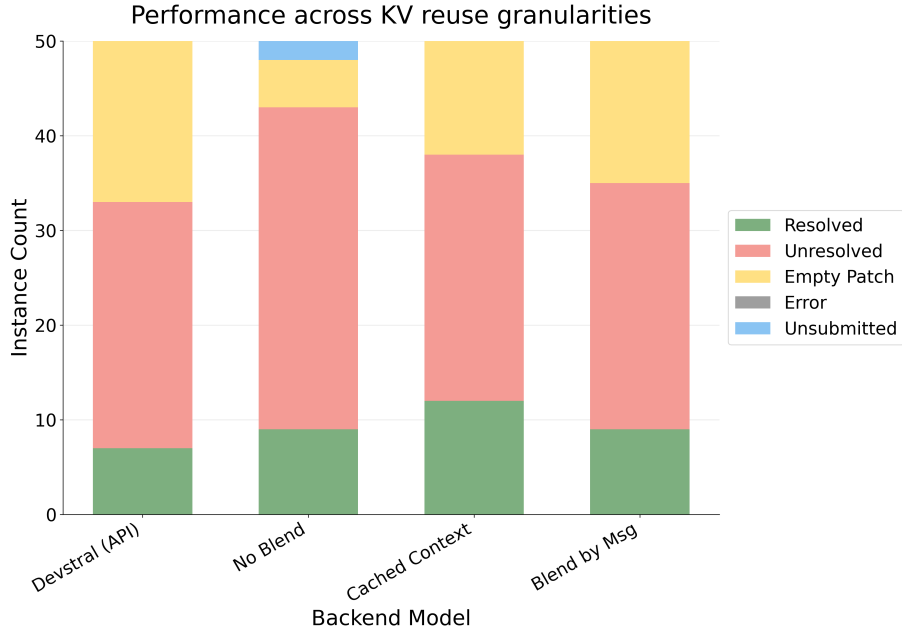


Figure 4.8: Resolve rate distributions for SWE-bench Verified-mini evaluation runs across different blending methods, served model is Devstral Small.

Bibliography

This bibliography contains 21 references.

- [Ant+25] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. *SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement*. 2025. arXiv: [2410.20285 \[cs.AI\]](#). URL: <https://arxiv.org/abs/2410.20285>.
- [Ant25] Anthropic. *Introducing Claude 4*. Announces Claude Code general availability. May 2025. URL: <https://www.anthropic.com/news/claude-4>.
- [Bai+23] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. *CodePlan: Repository-level Coding using LLMs and Planning*. 2023. arXiv: [2309.12499 \[cs.SE\]](#). URL: <https://arxiv.org/abs/2309.12499>.
- [Cho+24] Neil Chowdhury et al. *Introducing SWE-bench Verified*. 2024. URL: <https://openai.com/index/introducing-swe-bench-verified/>.
- [Gim+24] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. *Prompt Cache: Modular Attention Reuse for Low-Latency Inference*. 2024. arXiv: [2311.04934 \[cs.CL\]](#). URL: <https://arxiv.org/abs/2311.04934>.
- [Ho+20] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. *Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps*. 2020. arXiv: [2011.01060 \[cs.CL\]](#). URL: <https://arxiv.org/abs/2011.01060>.
- [Jia+25] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. *CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching*. 2025. arXiv: [2503.22424 \[cs.SE\]](#). URL: <https://arxiv.org/abs/2503.22424>.
- [Kwo+23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. arXiv: [2309.06180 \[cs.LG\]](#). URL: <https://arxiv.org/abs/2309.06180>.
- [LIB25] Lukas, Ian, and Balta. *Cursor Agent CLI*. <https://cursor.com/en/blog/cli>. Cursor Blog. Aug. 2025.
- [Mar25] MariusHobbhahn. *SWE-bench-verified-mini*. 2025. URL: <https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini>.

- [MS25] Taylor Mullen and Ryan J. Salva. *Introducing Gemini CLI: your open-source AI agent*. Google Developers Blog. 2025.
- [Ouy+25] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. *RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph*. 2025. arXiv: [2410.14684](https://arxiv.org/abs/2410.14684) [cs.SE]. URL: <https://arxiv.org/abs/2410.14684>.
- [SKS25] Nick Sharma, Mahi Kolla, and Ilai Soloducho. *Fully Reimagined: AI-First Google Colab*. Google Developers Blog. May 2025. URL: <https://developers.googleblog.com/en/fully-reimagined-ai-first-google-colab/>.
- [Tri+22] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. *MuSiQue: Multihop Questions via Single-hop Question Composition*. 2022. arXiv: [2108.00573](https://arxiv.org/abs/2108.00573) [cs.CL]. URL: <https://arxiv.org/abs/2108.00573>.
- [Xia+24] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. *Agentless: Demystifying LLM-based Software Engineering Agents*. 2024. arXiv: [2407.01489](https://arxiv.org/abs/2407.01489) [cs.SE]. URL: <https://arxiv.org/abs/2407.01489>.
- [Yan+24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. 2024. arXiv: [2405.15793](https://arxiv.org/abs/2405.15793) [cs.SE]. URL: <https://arxiv.org/abs/2405.15793>.
- [Yao+25] Jiayi Yao, Hanchen Li, Yuhua Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. *CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion*. 2025. arXiv: [2405.16444](https://arxiv.org/abs/2405.16444) [cs.LG]. URL: <https://arxiv.org/abs/2405.16444>.
- [Zha+24a] Lei Zhang, Yunshui Li, Jiaming Li, Xiaobo Xia, Jiayi Yang, Run Luo, Minzheng Wang, Longze Chen, Junhao Liu, and Min Yang. *Hierarchical Context Pruning: Optimizing Real-World Code Completion with Repository-Level Pretrained Code LLMs*. 2024. arXiv: [2406.18294](https://arxiv.org/abs/2406.18294) [cs.CL]. URL: <https://arxiv.org/abs/2406.18294>.
- [Zha+24b] Xinrong Zhang et al. ∞ Bench: Extending Long Context Evaluation Beyond 100K Tokens. 2024. arXiv: [2402.13718](https://arxiv.org/abs/2402.13718) [cs.CL]. URL: <https://arxiv.org/abs/2402.13718>.
- [Zha+24c] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. *AutoCodeRover: Autonomous Program Improvement*. 2024. arXiv: [2404.05427](https://arxiv.org/abs/2404.05427) [cs.SE]. URL: <https://arxiv.org/abs/2404.05427>.
- [Zhe+24] Lianmin Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. 2024. arXiv: [2312.07104](https://arxiv.org/abs/2312.07104) [cs.AI]. URL: <https://arxiv.org/abs/2312.07104>.